
Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission

Name	:	Jai Kousick T G
Register No.	:	192421367
Batch	:	2024
Degree / Branch	:	B.Tech Information Technology
Subject	:	DevSecOps Engineering and Application Security
Subject Code	:	CSA5802
Institution	:	SIMATS Engineering, Saveetha School of Engineering

1. Objective

The objective of this assessment is to design and document a complete DevSecOps architecture that integrates security controls at every stage of the software delivery lifecycle (Plan, Code, Build, Test, Release, Deploy, Operate, Monitor), to document the methodology and tooling used to enforce these controls, and to define a GitHub repository structure and CI/CD workflow suitable for submitting and continuously validating the project.

2. Scope of the Deliverable

This report covers three deliverables required by the assessment: (1) a complete DevSecOps architecture diagram showing the pipeline stages and the security control integrated into each; (2) methodology documentation explaining the shift-left security approach, phase-wise activities, threat modeling method, security gate criteria, and tool rationale; and (3) a GitHub submission structure, including the repository layout, branching strategy, a working CI/CD pipeline definition (GitHub Actions), and README/commit conventions.

3. Complete DevSecOps Architecture

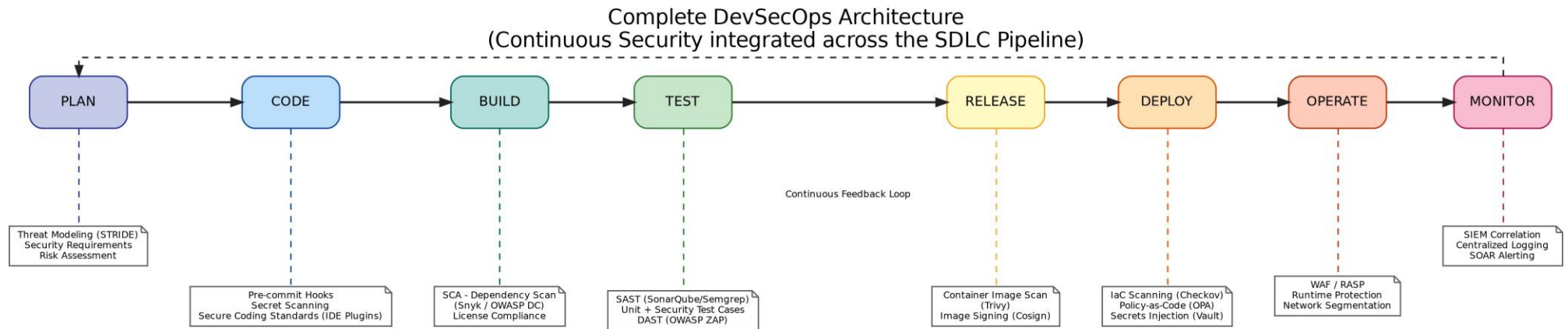


Figure 1: Complete DevSecOps architecture — the Plan-Code-Build-Test-Release-Deploy-Operate-Monitor pipeline with the security control(s) integrated at each stage and the continuous feedback loop back into planning.

4. Stage-wise Architecture Description

Plan

Requirements and threat modeling stage. Security requirements are captured alongside functional requirements; STRIDE-based threat modeling and risk assessment identify likely attack surfaces before a single line of code is written.

Code

Developers work in an IDE fitted with security linters; pre-commit hooks block commits containing hard-coded secrets (API keys, passwords); secure coding standards (input validation, output encoding, parameterized queries) are enforced via code review checklists.

Build

As dependencies are pulled in, Software Composition Analysis (SCA) tools scan for known vulnerable libraries (CVEs) and license-compliance issues before the build artifact is produced.

Test

Static Application Security Testing (SAST) analyzes source code for insecure patterns; unit and dedicated security test cases run automatically; Dynamic Application Security Testing (DAST) exercises a running instance of the application to catch runtime issues SAST cannot see.

Release

Container images are scanned for OS and package-level vulnerabilities before being pushed to a registry; images are then cryptographically signed so that only verified, untampered images can be deployed.

Deploy

Infrastructure-as-Code (IaC) templates are scanned for misconfigurations; Policy-as-Code gates block deployments that violate organizational security policy; secrets are injected at deploy time from a dedicated secrets manager rather than being baked into artifacts.

Operate

Running workloads are protected by a Web Application Firewall (WAF) and, where applicable, Runtime Application Self-Protection (RASP); network segmentation limits blast radius if a component is compromised.

Monitor

All tiers stream events to centralized, immutable logging; a SIEM correlates events across the pipeline and production environment; a SOAR-integrated alerting layer notifies the security team, and findings feed back into the Plan stage, closing the continuous feedback loop.

5. Methodology Documentation

5.1 Methodology Adopted: Shift-Left Secure SDLC

The project follows a **shift-left** security methodology: security activities that are traditionally performed only just before release (penetration testing, manual security review) are moved as early as possible into the planning, coding, and build stages. This reduces the cost of fixing vulnerabilities (a defect found at Plan/Code stage is orders of magnitude cheaper to fix than one found in Production) and ensures security is a continuous, automated activity rather than a one-time gate before release.

5.2 Threat Modeling Approach — STRIDE

Threat Category	Description	Primary Mitigation
Spoofing	Impersonating another user or system	Strong authentication, MFA, mutual TLS
Tampering	Unauthorized modification of data/code	Code signing, integrity checks, IaC scanning
Repudiation	Denying an action was performed	Centralized immutable audit logging
Information Disclosure	Exposing data to unauthorized parties	Encryption at rest/in transit, RBAC
Denial of Service	Making a system unavailable	Rate limiting, WAF, anti-DDoS at the edge
Elevation of Privilege	Gaining higher access than granted	Least-privilege IAM, RBAC, policy-as-code

5.3 Phase-wise Activities, Tools and Ownership

Phase	Key Activity	Tool(s)	Owner
Plan	Threat modeling, risk assessment, security requirements	STRIDE, DFD tools	Security + Product
Code	Secure coding, secret scanning, peer review	Gitleaks, IDE SAST plugins	Developers
Build	Dependency / license scanning	OWASP Dependency-Check, Snyk	Dev + CI
Test	SAST, unit/security tests, DAST	CodeQL, Semgrep, OWASP ZAP	QA + Security
Release	Container scan, image signing	Trivy, Cosign	DevOps
Deploy	IaC scan, policy-as-code, secret injection	Checkov, OPA/Conftest, Vault	DevOps + Security
Operate	Runtime protection, network segmentation	WAF, RASP, firewalls	SRE / Ops
Monitor	Log correlation, alerting, incident response	SIEM, SOAR	SOC Team

5.4 Security Gate Criteria

Each stage of the pipeline enforces an automated quality/security gate. A build is not permitted to progress to the next stage unless it satisfies the following criteria:

Gate	Pass Criteria
Secret Scan Gate	Zero verified secrets detected in the diff being committed.
SCA Gate	No dependency with a Critical or High severity CVE and no available patch.
SAST Gate	No Critical severity finding; High severity findings triaged and either fixed or formally risk-accepted by the security owner.
Test Gate	Unit + security test coverage above the agreed threshold (e.g. 80%); all security test cases pass.
Image Scan Gate	No Critical/High vulnerability in the base image or application layer; image must be signed before deployment is permitted.
IaC / Policy Gate	No IaC misconfiguration flagged as Critical/High; all Policy-as-Code (OPA) rules evaluate to allow.
Production Approval Gate	Manual reviewer approval required on the protected 'production' GitHub environment in addition to all automated gates passing.

5.5 Compliance and Standards Mapping

The methodology is mapped to recognized industry frameworks so that the controls chosen are not ad-hoc: application-level findings are triaged against the **OWASP Top 10**; the overall secure development process aligns to the **NIST Secure Software Development Framework (SSDF, SP 800-218)**; and infrastructure/container baselines are checked against relevant **CIS Benchmarks**.

6. GitHub Repository Structure & Submission

6.1 Repository Folder Structure

```
devsecops-project/
|-- .github/
|   |-- workflows/
|       |-- devsecops-pipeline.yml      # CI/CD pipeline with integrated security gates
|-- docs/
|   |-- architecture.md                # Architecture diagram + description
|   |-- methodology.md                 # This methodology documentation
|   |-- threat-model.md                # STRIDE threat model
|-- infra/
|   |-- terraform/                     # Infrastructure-as-Code (scanned by Checkov)
|   |-- kubernetes/                    # K8s manifests (scanned by Checkov)
|-- policies/
|   |-- opa/                           # Policy-as-Code rules (OPA/Conftest)
|-- src/
|   |-- app/                           # Application source code
|   |-- tests/                         # Unit + security test cases
|-- security/
|   |-- sast-config/                   # SonarQube / Semgrep rule configs
|   |-- reports/                       # Generated SARIF / scan reports (gitignored)
|-- .gitignore.toml                    # Secret-scanning rules
|-- .gitignore
|-- Dockerfile
|-- README.md
-- LICENSE
```

6.2 Branching Strategy

A trunk-based branching strategy with protected branches is used: **main** (production-ready, protected, requires PR review + all security gates green + manual approval), **develop** (integration branch, deploys automatically to staging once gates pass), and short-lived **feature/*** and **hotfix/*** branches created from and merged back via Pull Request. Branch protection rules require at least one approving review, all required status checks (secret-scan, sca-scan, sast-scan, unit-tests) to pass, and signed commits before merging into main or develop.

6.3 CI/CD Pipeline Definition (GitHub Actions)

The following `.github/workflows/devsecops-pipeline.yml` implements the architecture in Section 3 as an executable GitHub Actions workflow, with a job for each pipeline stage and its corresponding security control:

```
name: DevSecOps CI/CD Pipeline

# Trigger: runs on every push and pull request targeting main/develop
on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main", "develop" ]

permissions:
  contents: read
  security-events: write      # required to upload SARIF results to GitHub code scanning

jobs:

  # -----
  # 1. CODE STAGE - Secret scanning + lint
  # -----
  secret-scan:
    name: Secret Scanning (Gitleaks)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```

    with:
      fetch-depth: 0
  - name: Run Gitleaks
    uses: gitleaks/gitleaks-action@v2
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

# -----
# 2. BUILD STAGE - Software Composition Analysis (SCA)
# -----
sca-scan:
  name: Dependency / SCA Scan
  runs-on: ubuntu-latest
  needs: secret-scan
  steps:
    - uses: actions/checkout@v4
    - name: Run OWASP Dependency-Check
      uses: dependency-check/Dependency-Check_Action@main
      with:
        project: "devsecops-app"
        path: "."
        format: "SARIF"
        out: "reports"
    - name: Upload SCA results
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: reports/dependency-check-report.sarif

# -----
# 3. TEST STAGE - Static Application Security Testing (SAST)
# -----
sast-scan:
  name: SAST (CodeQL / Semgrep)
  runs-on: ubuntu-latest
  needs: sca-scan
  steps:
    - uses: actions/checkout@v4
    - name: Initialize CodeQL
      uses: github/codeql-action/init@v3
      with:
        languages: javascript, python
    - name: Autobuild
      uses: github/codeql-action/autobuild@v3
    - name: Perform CodeQL Analysis
      uses: github/codeql-action/analyze@v3
    - name: Run Semgrep
      uses: returntocorp/semgrep-action@v1
      with:
        config: p/owasp-top-ten

unit-tests:
  name: Unit + Security Test Cases
  runs-on: ubuntu-latest
  needs: sca-scan
  steps:
    - uses: actions/checkout@v4
    - name: Set up runtime
      uses: actions/setup-node@v4
      with:
        node-version: "20"
    - name: Install dependencies
      run: npm ci
    - name: Run unit + security test suite
      run: npm test----- coverage

# -----
# 4. RELEASE STAGE - Container build, image scan, and signing
# -----
build-and-scan-image:
  name: Build Image, Trivy Scan, Cosign Sign
  runs-on: ubuntu-latest
  needs: [ sast-scan, unit-tests ]
  steps:
    - uses: actions/checkout@v4
    - name: Build container image
      run: docker build -t ghcr.io/${ github.repository }:${ github.sha } .
    - name: Scan image with Trivy
      uses: aquasecurity/trivy-action@master
      with:

```



```

        image-ref: "ghcr.io/${{ github.repository }}:${{ github.sha }}"
        format: "sarif"
        output: "trivy-results.sarif"
        severity: "CRITICAL,HIGH"
        exit-code: "1"
    - name: Upload Trivy results
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: trivy-results.sarif
    - name: Install Cosign
      uses: sigstore/cosign-installer@v3
    - name: Sign container image
      run: cosign sign --yes ghcr.io/${{ github.repository }}:${{ github.sha }}

# -----
# 5. DEPLOY STAGE - IaC scanning + policy-as-code gate
# -----
iac-scan:
  name: IaC Scan (Checkov) + OPA Policy Gate
  runs-on: ubuntu-latest
  needs: build-and-scan-image
  steps:
    - uses: actions/checkout@v4
    - name: Run Checkov on Terraform/K8s manifests
      uses: bridgecrewio/checkov-action@master
      with:
        directory: infra/
        framework: terraform,kubernetes
        output_format: sarif
        output_file_path: checkov-results.sarif
    - name: Upload Checkov results
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: checkov-results.sarif
    - name: Evaluate OPA/Conftest policies
      run: |
        conftest test infra/ --policy policies/

deploy-staging:
  name: Deploy to Staging + DAST
  runs-on: ubuntu-latest
  needs: iac-scan
  if: github.ref == 'refs/heads/develop'
  environment: staging
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to staging environment
      run: echo "Deploying image ${{ github.sha }} to staging..."
    - name: DAST scan with OWASP ZAP
      uses: zapproxy/action-full-scan@v0.10.0
      with:
        target: "https://staging.example.com"

deploy-production:
  name: Deploy to Production (Manual Approval Gate)
  runs-on: ubuntu-latest
  needs: iac-scan
  if: github.ref == 'refs/heads/main'
  environment: production # protected environment requires manual reviewer approval
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to production environment
      run: echo "Deploying image ${{ github.sha }} to production..."

# -----
# 6. OPERATE / MONITOR - post-deploy verification
# -----
post-deploy-monitor:
  name: Post-Deploy Health & Security Check
  runs-on: ubuntu-latest
  needs: [ deploy-staging, deploy-production ]
  if: always()
  steps:
    - name: Notify SIEM / logging pipeline
      run: echo "Forwarding deployment event and logs to centralized SIEM..."

```

6.4 README.md Outline

- Project title, one-line description, and architecture diagram image
- Badges: build status, security scan status, code coverage
- Prerequisites and local setup instructions
- How to run the application and the test suite locally
- Link to docs/architecture.md and docs/methodology.md
- Security policy: how to report a vulnerability (SECURITY.md)
- Contribution guidelines and branching strategy summary
- License

6.5 Commit and Pull Request Conventions

Commits follow the **Conventional Commits** format (e.g. `feat:`, `fix:`, `sec:`, `chore:`) so that security-relevant fixes are traceable. Every Pull Request description must state which security gates were affected and link to the corresponding SARIF/scan report generated by the workflow. Releases are tagged using Semantic Versioning (e.g. v1.2.0) and each tag triggers the release job that produces a signed, scanned container image.

7. Conclusion

This report has presented a complete DevSecOps architecture integrating security controls across every stage of the pipeline — from threat modeling at Plan through to SIEM-driven monitoring in Operate/Monitor — supported by a documented shift-left methodology, STRIDE-based threat modeling, phase-wise tool ownership, and explicit automated security gate criteria mapped to OWASP, NIST SSDF and CIS standards. The accompanying GitHub repository structure, branch protection strategy, and a working GitHub Actions CI/CD workflow operationalize this architecture, ensuring that security is enforced continuously and automatically rather than as a one-time, end-of-cycle check.